Report notes

Github Link

These notes describe the final measurements (`results/q1-linux-30s`, `results/q2-linux`, `results/q3`). Reproduction commands and handout question/page references are in README.md.

Measurement method (Q1(c), Q2(a))

The OS default is read once, applied to every data socket before connecting, and queried afterward. Every successful saved run used **cubic**. The Python client implements the iPerf3 exchange; the kernel implements TCP and CUBIC.

TCP_INFO uses native byte order with explicit byte offsets: `=I` is an unsigned 32-bit value and `=Q` an unsigned 64-bit value. We request 256 bytes and require at least 128 for `bytes_acked`; unavailable optional values are null/blank. The kernel, byte order, layout, and actual returned length are recorded with runs.

Field	Offset	Format	Unit
snd_mss	16	=I	bytes
rtt	68	=I	microseconds, already unscaled
rttvar	72	=I	microseconds
snd_cwnd	80	=I	segments
total_retrans	100	=I	cumulative segments
pacing_rate	104	=Q	bytes/s
bytes_acked	120	=Q	cumulative bytes
min_rtt	148	=I	microseconds, kernel windowed minimum (added for Q3; absent in the Docker runs)
delivery_rate	160	=Q	bytes/s, optional

Layout reference: Linux TCP_INFO. Unlimited pacing values and an unavailable zero delivery-rate estimate become null. JSON retains any additional fields recorded by earlier versions of the sampler.

```
payload_acked = max(0, bytes_acked - pre_cookie_bytes_acked - 37)
dt = current_monotonic_time - previous_monotonic_time
goodput_bps = 8 * delta(payload_acked) / dt
loss_proxy = delta(total_retrans)              # retransmitted segments / interval
```

The 37-byte iPerf3 cookie is excluded even if its ACK arrives late. Baseline, periodic, and final snapshots use the live upload socket. The baseline has no interval rate; all subsequent intervals, including startup, zeros, and the final partial interval, are included. No drain period is measured after sending ends. Timestamp is UTC; `_seconds`, `_us`, `_bps`, and byte/segment field names give units. RTT/cwnd are endpoint snapshots; goodput/loss describe the preceding interval.

`summary.csv` groups successful intervals by destination, port, and actual algorithm. Mean is the unweighted mean of interval rates; p95 interpolates at `0.95*(N-1)` in sorted rates. Overall goodput uses total

acknowledged bytes divided by total measured time. Failed/interrupted attempts are excluded from summaries. Rates are decimal Mbit/s. This Q1/Q2 run plan has one repetition per destination.

Metric meanings and expected effects (Q2(c))

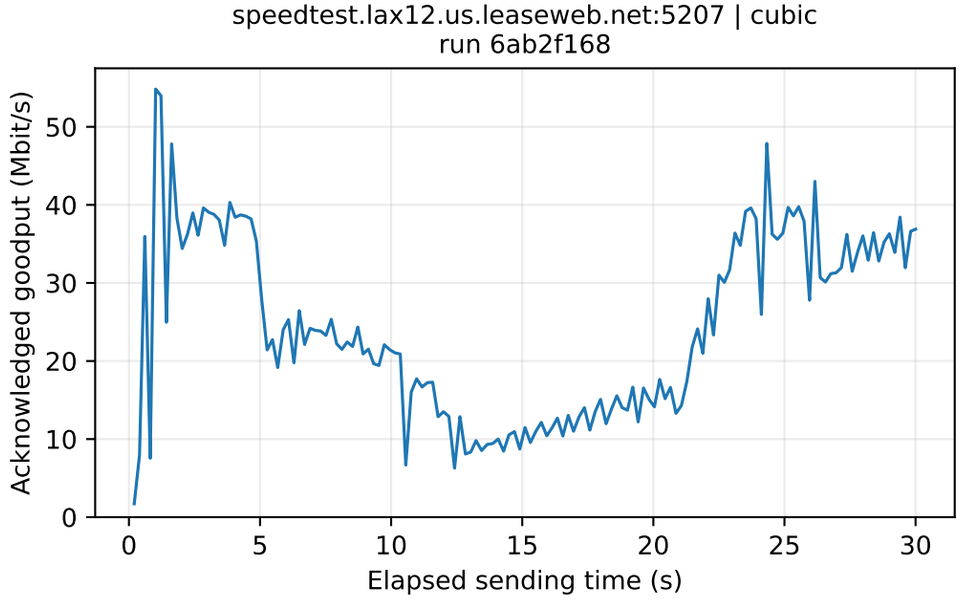
- **cwnd:** sender congestion allowance in segments. When window limited, a larger window can raise goodput; the rough bound is `8*cwnd*MSS/RTT_seconds`. It is an allowance, not the amount currently in flight.
- **RTT:** smoothed round-trip time. At fixed cwnd, longer RTT slows ACK feedback and lowers the window-limited rate. Rising RTT with flat goodput can suggest queues.
- **Loss proxy:** retransmitted segments per interval. Recovery can delay delivery and reduce cwnd. Retransmissions can be repeated or spurious; this is not a loss percentage, and loss detection can lag its effects.
- **Goodput:** newly acknowledged useful payload per actual elapsed second. Retransmitted payload counts once when acknowledged. Local `send()` acceptance and the kernel's latest delivery-rate estimate are different measurements.

Q1/Q2 results and observations

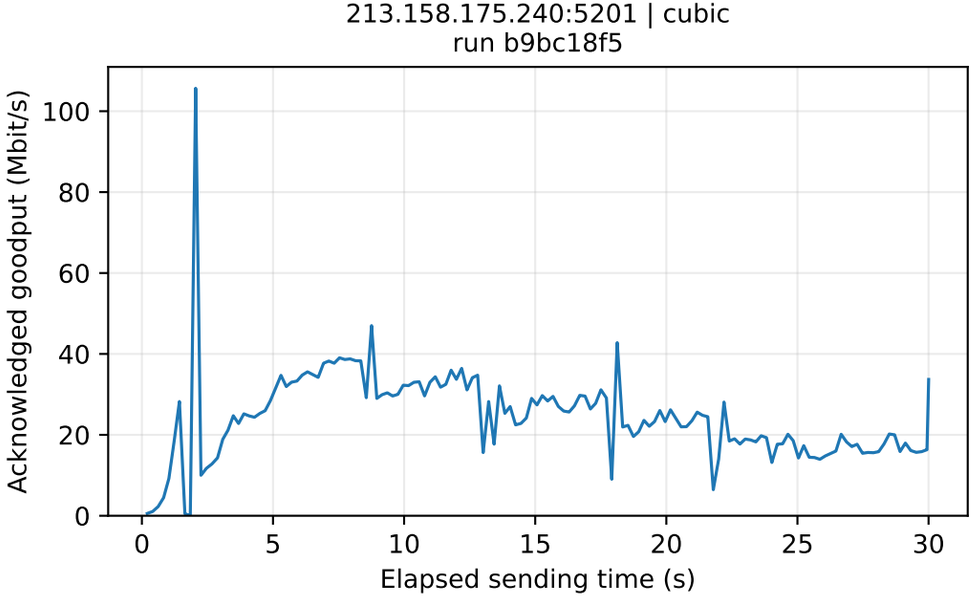
Environment. Final Q1/Q2 data comes from the same host as Q3: an Ubuntu 26.04.1 (ARM64) live session in a UTM VM on an Apple M3 Mac, kernel `7.0.0-30-generic`, Python 3.14.4, run natively (not in Docker), with UTM Shared Network (NAT), on 2026-10-18. The OS default and every run is **cubic**. An earlier run under Docker Desktop on macOS (kernel `6.10.14-linuxkit`) was discarded. Docker Desktop's networking ends the container's TCP connection at a local proxy, so those runs measured the hop to that proxy, not the path to the server: MSS 65483 bytes, RTT 0.2–16 ms to Los Angeles, cwnd fixed near 40, no retransmissions, and goodput up to 256 Mbit/s, which is more than the ≈ 45 Mbit/s uplink can carry (see Q3). In the Linux runs MSS is 1448 bytes (1368–1424 on two paths), and RTTs match distance.

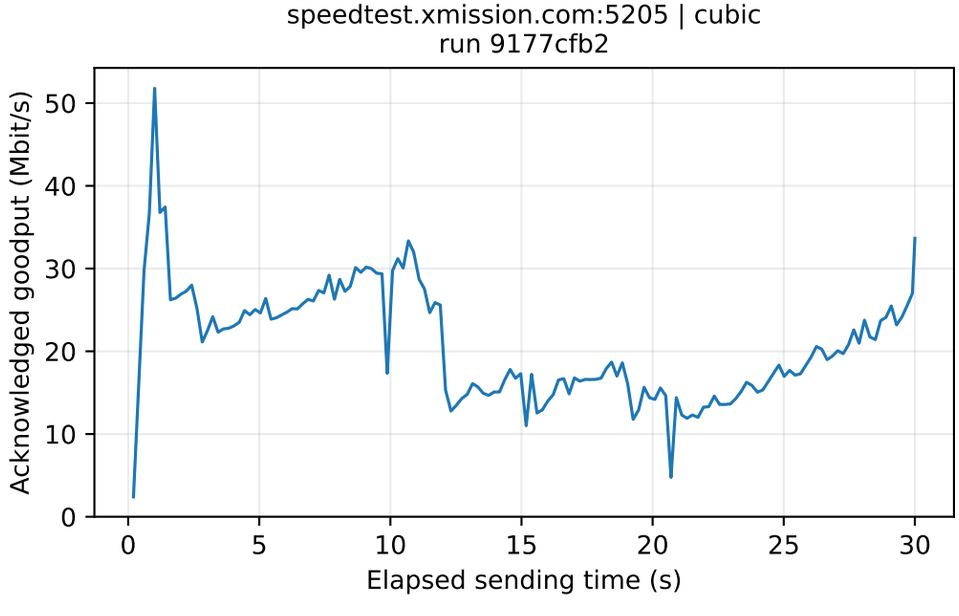
Q1: goodput plots, summary table. Ten destinations completed in 15 attempts, with 30-second uploads, 0.2-second sampling, and seed 422: 1470 measured intervals. Failures were kept and handled as specified: `iperf.pendc.com` (reset during upload, then timeout on retry) and `185.180.12.40` (`ACCESS_DENIED`, server busy, twice) were replaced; `96.45.38.22` closed the data connection once and succeeded on retry. No shortfall.

Q1: 10/10 destinations completed

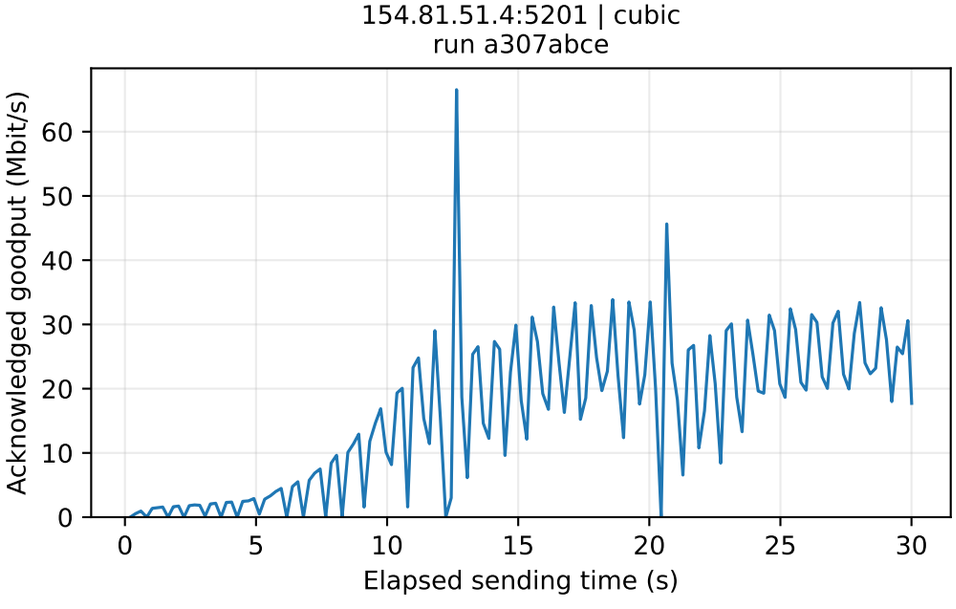


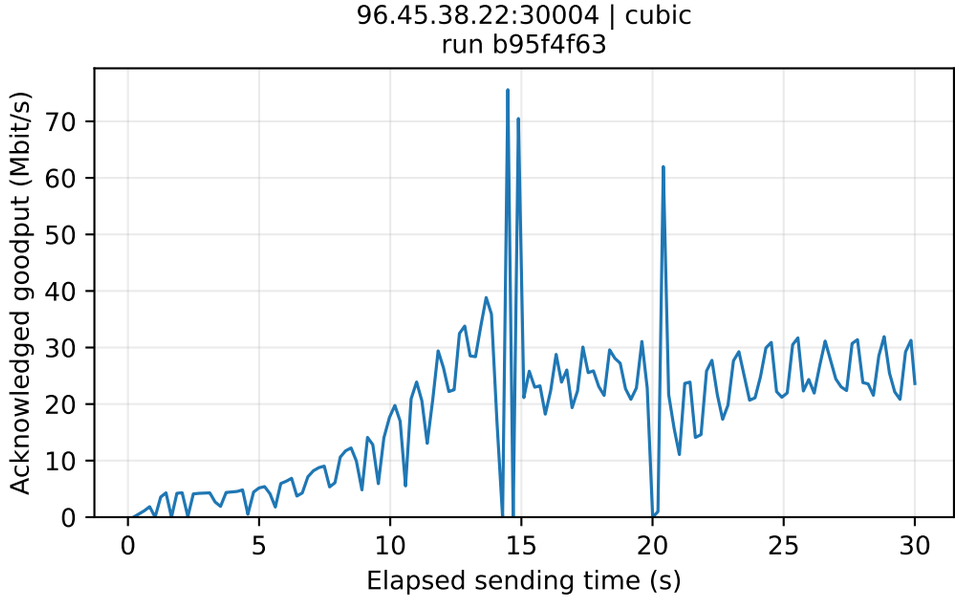


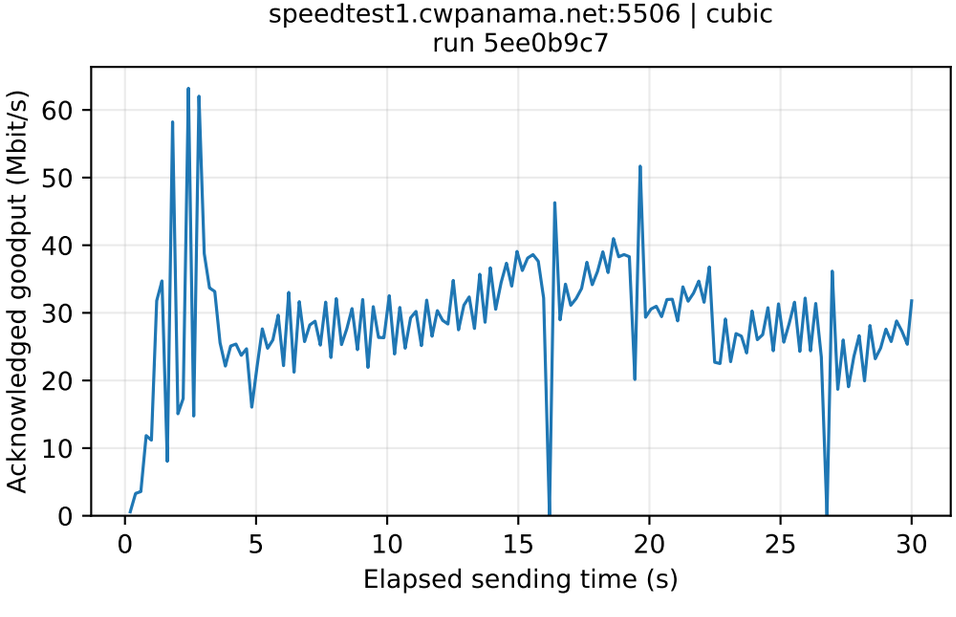


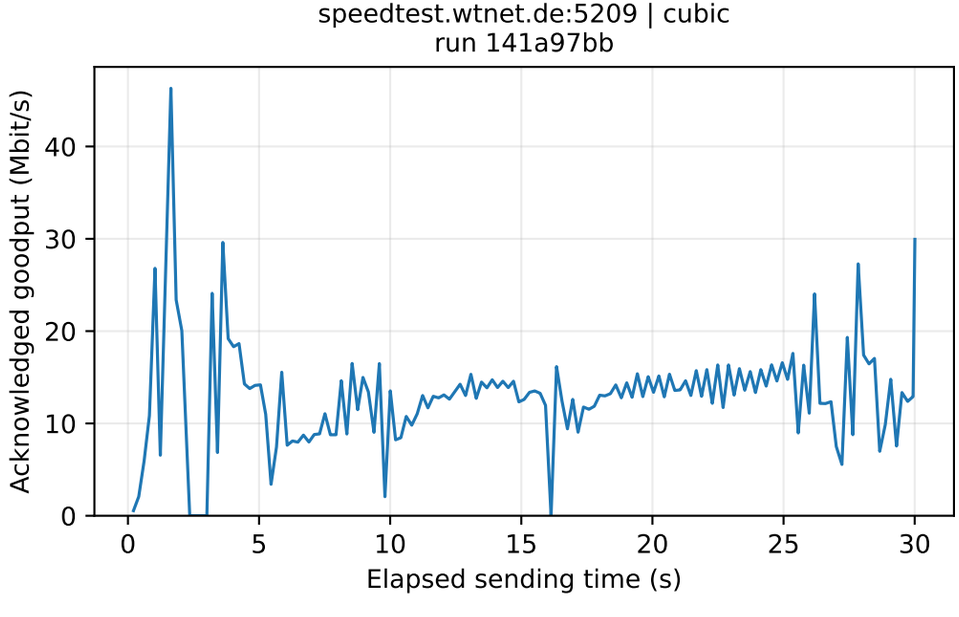


Q1: 10/10 destinations completed

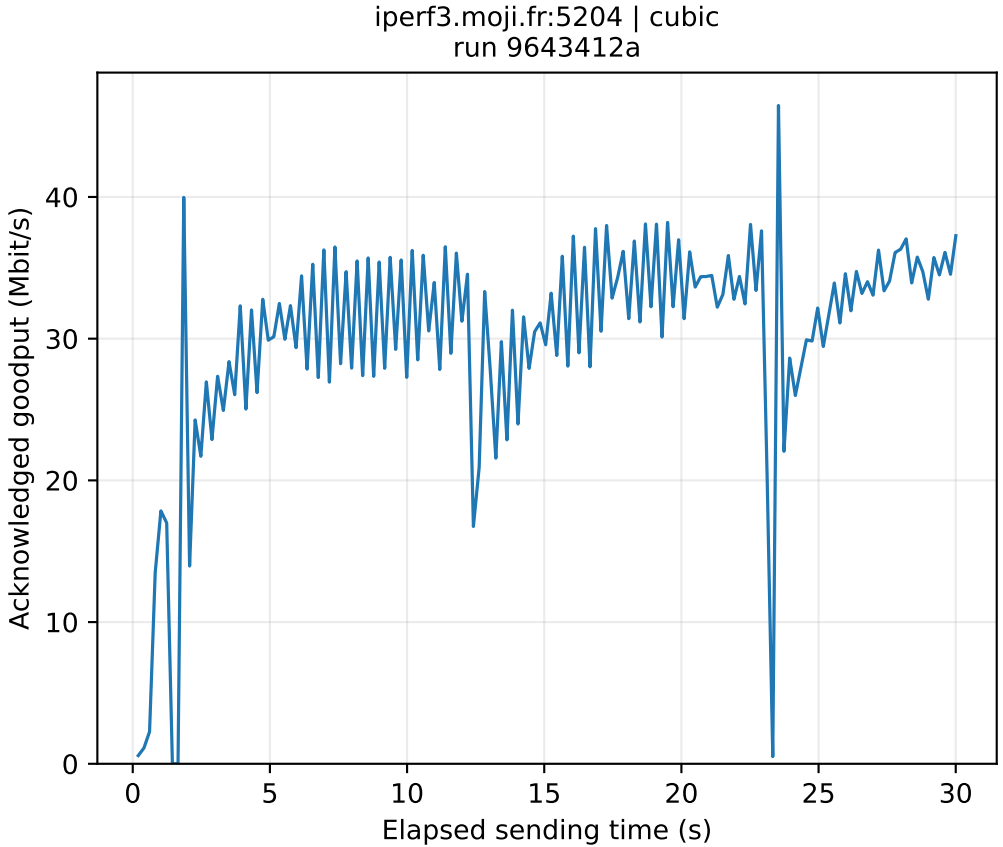


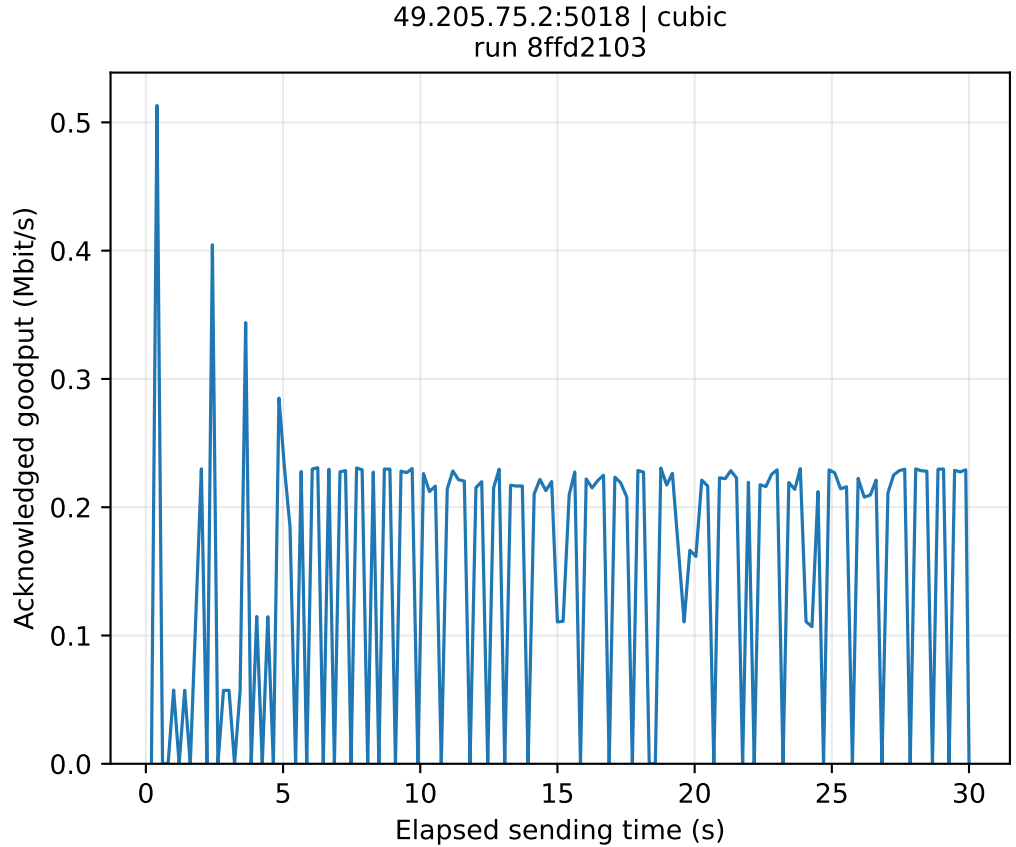






Q1: 10/10 destinations completed





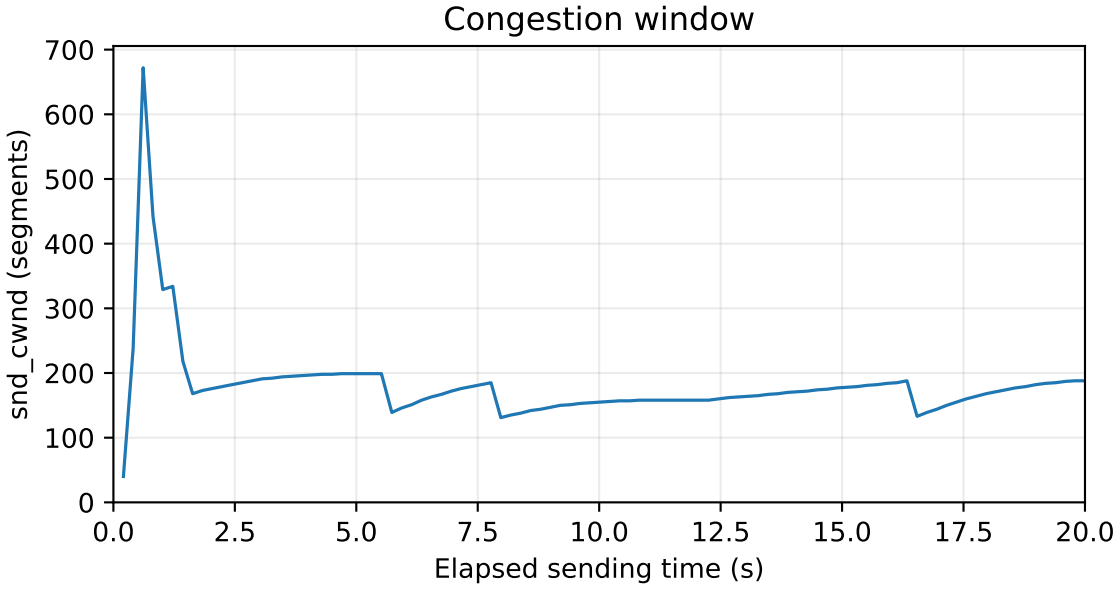
Destination	min RTT (ms)	Overall goodput (Mbit/s)	Median / p95 interval goodput
iperf3.moji.fr (France)	125	30.1	32.2 / 37.7
speedtest1.cwpanama.net (Panama)	115	28.7	28.9 / 38.9
speedtest.hkg12.hk.leaseweb.net (Hong Kong)	230	28.3	30.3 / 39.0
213.158.175.240	173	24.3	24.4 / 38.3
speedtest.lax12.us.leaseweb.net (Los Angeles)	68	24.2	22.4 / 39.7
speedtest.xmission.com (Salt Lake City)	56	20.8	19.6 / 30.8
96.45.38.22	244	18.4	21.2 / 32.3
154.81.51.4	259	16.3	18.1 / 32.9
speedtest.wtnet.de (Germany)	127	12.9	13.1 / 23.9
49.205.75.2 (India)	254	0.2	0.2 / 0.2

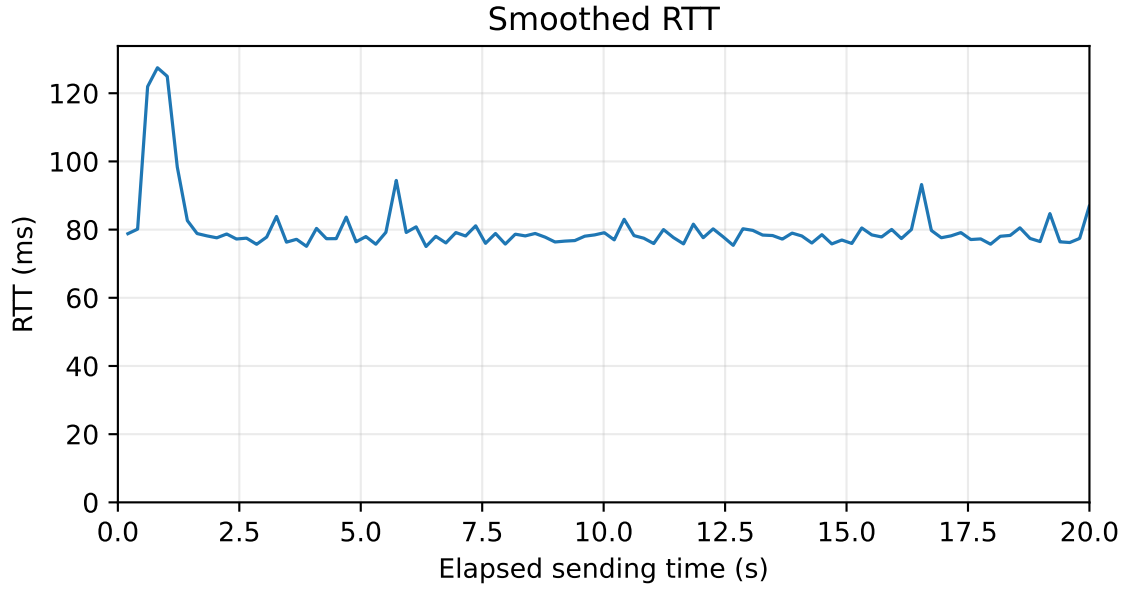
Observations:

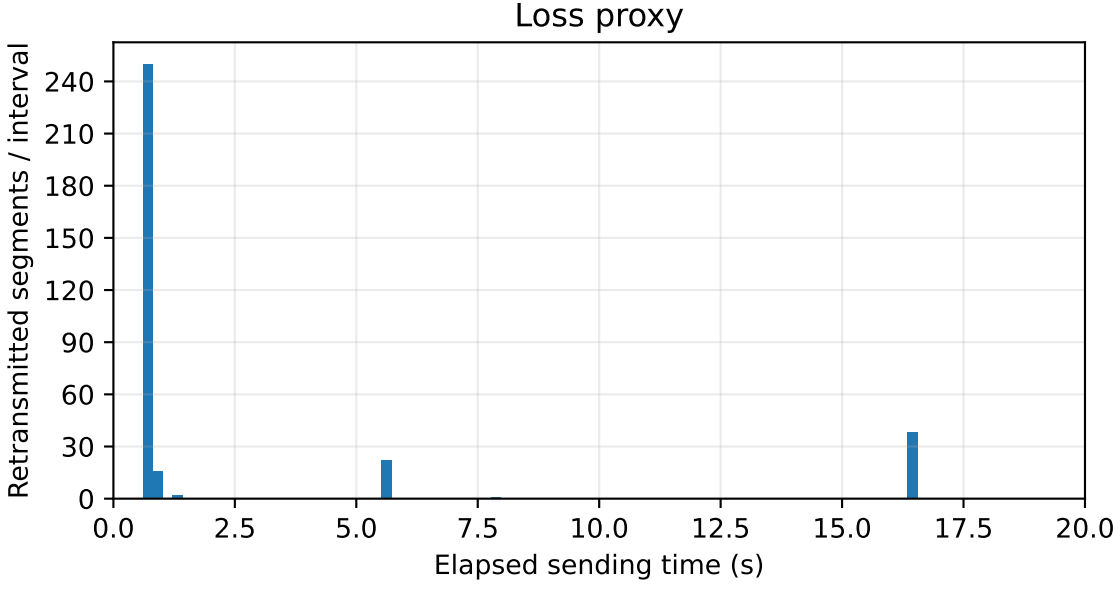
- *A common ceiling.* p95 interval goodput is 31–40 Mbit/s for nine of ten destinations, whatever the distance: the same ≈ 45 Mbit/s bottleneck near the sender found in Q3. Average goodput is below that ceiling because CUBIC backs off after each loss.
- *Upload length matters on long paths.* Slow start doubles cwnd once per RTT, so it reaches the path's window size much later on a 250 ms path than on a 60 ms one. A 3-second trial run (discarded) left the 240–400 ms paths at 0.7–2 Mbit/s, still in slow start, which is why the final run uses 30 s uploads. Even at 30 s the start-up cost is visible: for 154.81.51.4 and 96.45.38.22 the average rate after the first 10 s is about 22–25 Mbit/s, higher than their 30 s averages (16–18 Mbit/s).
- *Per-path differences come from loss, not distance alone.* Hong Kong (230 ms) averaged more than Los Angeles (68 ms) in this run; the Q3 repetitions show that single CUBIC runs vary by 5–47%, so a one-run ranking of destinations is not reliable. Germany (12.9 Mbit/s) had 610 retransmissions, more than most paths, which keeps its window lower.
- *Anomaly: 49.205.75.2.* This path stayed at 0.2 Mbit/s for the whole 30 s, with cwnd never above 19 segments and RTT up to ≈ 290 ms. A window that small and flat suggests a limit outside our sender's congestion control (a server- or network-side rate limit, or a very lossy path), but these measurements cannot identify the cause.
- Every successful interval is included, so `min_goodput_mbps` is 0.0 for most destinations: the last sample covers a sub-millisecond interval after sending stops.

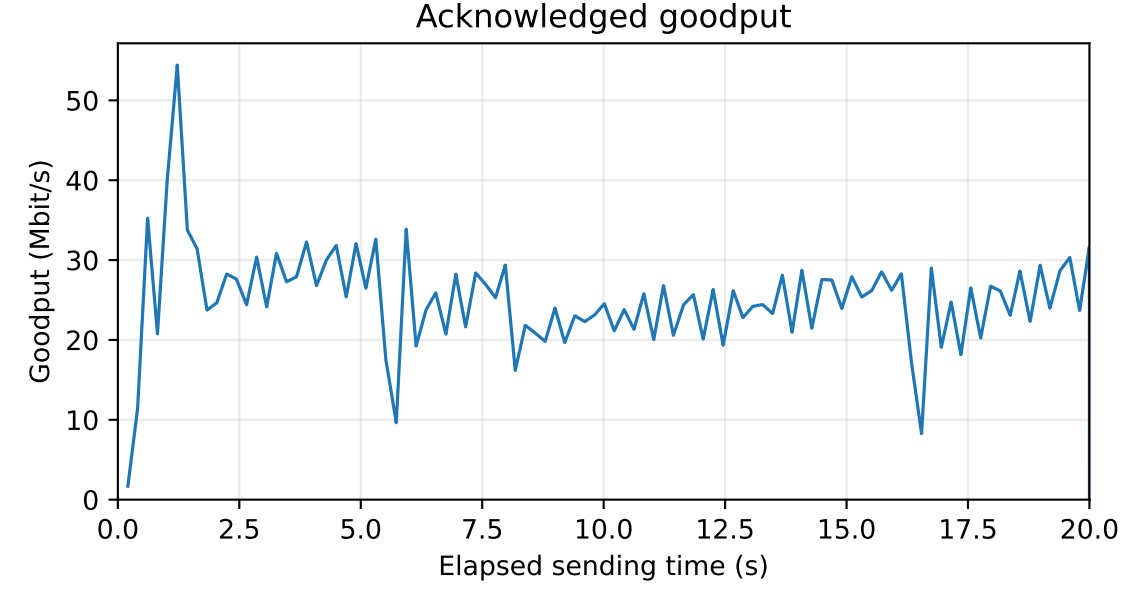
Q2: time series, scatter plots. Representative destination: `speedtest.lax12.us.leaseweb.net:5207` (the first successful destination; the same server as the Docker Q2 run); run `cac92fef-6d6d-4d01-8073-75a996b32d68`, **cubic**, 99 intervals over 20.0 s, seed 422. Overall goodput was **25.0 Mbit/s**. Every Q2 figure and observation uses this run.

Q2: TCP statistics during upload
speedtest.lax12.us.leaseweb.net:5207 | cubic | run cac92fef



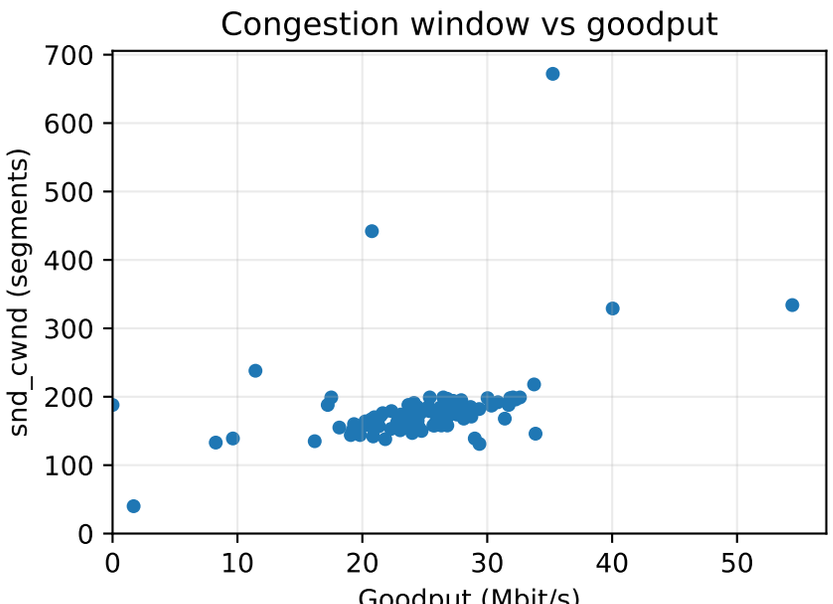


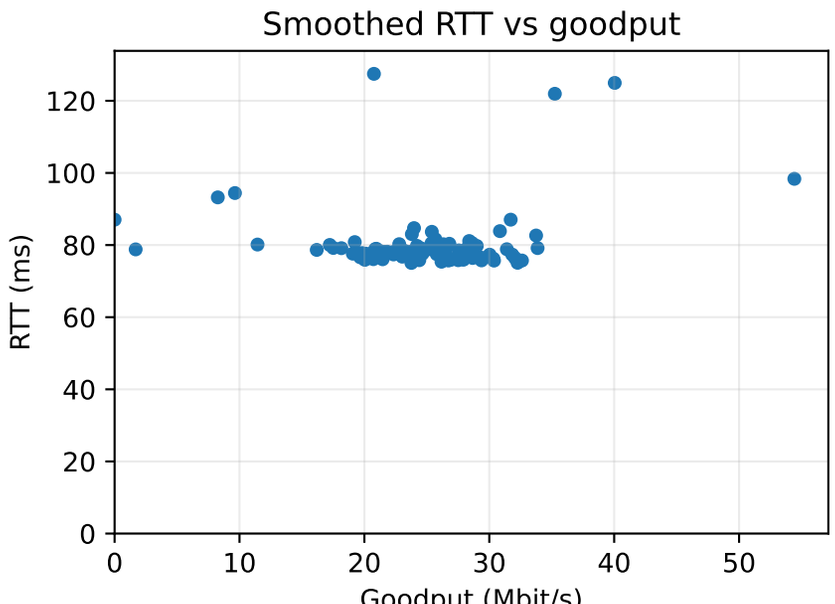


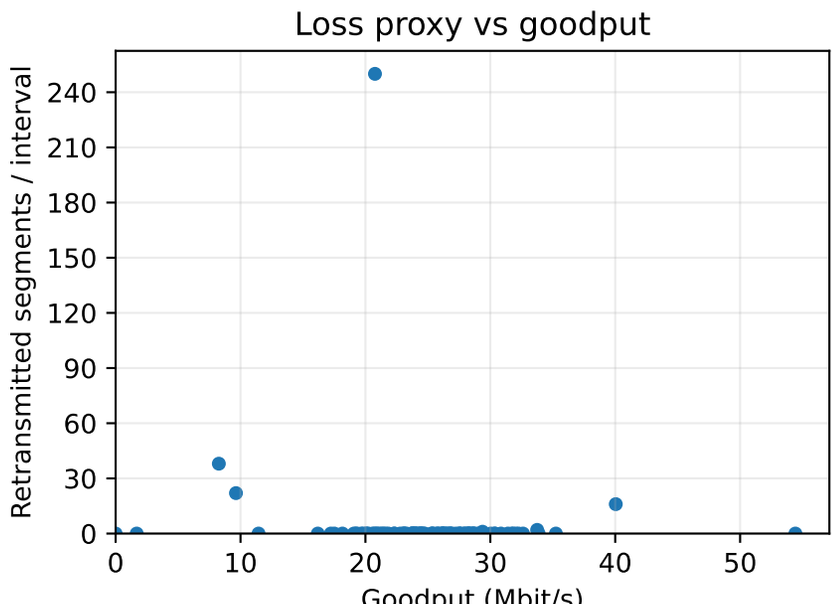


cwnd/RTT are endpoint snapshots; goodput/loss cover the preceding interval.

Q2: TCP metrics and goodput
speedtest.lax12.us.leaseweb.net:5207 | cubic | run cac92fef







Each point uses the same sampling interval. Correlation does not establish causation.

Metric	Minimum	Median	Maximum
cwnd (segments)	40	172	672
Smoothed RTT (ms)	75.0	78.1	127.5
Retransmitted segments / interval	0	0	250
Goodput (Mbit/s)	0.0 (final partial interval)	25.3	54.4

Kernel minimum RTT was 69.6 ms, so the median smoothed RTT is ≈ 8.5 ms above the path's propagation delay.

Observations (Q2(c)):

- *Slow start and overshoot.* cwnd grows from 40 to 672 segments in the first 0.6 s. That exceeds the BDP (≈ 45 Mbit/s × 70 ms ≈ 270 segments) plus the small buffer, so the queue overflows: RTT rises to 127.5 ms, 250 segments are retransmitted in one interval (0.82 s), and cwnd falls to ≈ 170. Goodput peaks at 54 Mbit/s during the overshoot, as queued data drains.
- *Congestion avoidance follows the CUBIC curve.* After 1.5 s cwnd grows quickly, then flattens near its previous maximum (≈ 200 segments), then grows again: CUBIC's concave-then-convex window function. Goodput tracks cwnd at ≈ 25–30 Mbit/s, consistent with `8·cwnd·MSS/RTT` (172 × 1448 × 8 / 0.078 s ≈ 25.5 Mbit/s at the medians).
- *Loss events cause the goodput drops.* At 5.73 s, 22 segments were retransmitted, cwnd fell from 199 to 139 (×0.70, CUBIC's β), RTT rose from 79 to 94 ms, and interval goodput fell to 9.6 Mbit/s. At 16.54 s, 38 retransmissions cut cwnd from 188 to 133 (×0.71), RTT rose to 93 ms, and goodput fell to 8.3 Mbit/s. In both, RTT spikes in the loss interval (the queue had filled) and recovers within one interval after the cut.
- *RTT is flat in between.* Between losses the queue stays small (smoothed RTT 75–85 ms against a 69.6 ms minimum), so on this path the cwnd and loss signals explain goodput better than RTT does. The scatter plots show this: cwnd and goodput are positively related, while RTT and the loss proxy are near-constant except at the few loss intervals.

Scatter associations do not establish causation. Loss is measured as retransmitted segments, which can include spurious retransmissions. One 20 s run on a public path with uncontrolled home traffic cannot establish repeatability; Q3 adds repetitions.

Q3(a): CUBIC, Reno, and BBR comparison

Data: results/q3. The saved Q1/Q2 data could not be reused: Docker Desktop's kernel offered only `reno cubic` and distorted cwnd/MSS.

Method. Same destinations, duration, and sampling interval for every algorithm. The algorithm is set with `TCP_CONGESTION` on the data socket before `connect()` and read back afterward (`actual_algorithm`). Repetition 1 admits destinations: a server counts only if it completes a test with all three algorithms; otherwise it is logged and replaced. Each repetition rotates the order (cubic,reno,bbr / reno,bbr,cubic / bbr,cubic,reno), so no algorithm always runs first while the path drifts. Repetitions 2..R sweep all admitted destinations, so repeats are minutes apart. Later failures are reported as missing runs, not retried.

Run. 5 destinations × 3 algorithms × 3 repetitions = 45 uploads of 30 s each, 0.2 s sampling, seed 422, 5 s pause before each test, at most 1 retry; 2026-10-18 21:19–21:47 UTC. All 5 first-choice destinations were admitted and all 45 runs completed; the only 2 failures (Hong Kong, `premature EOF` at the start of the

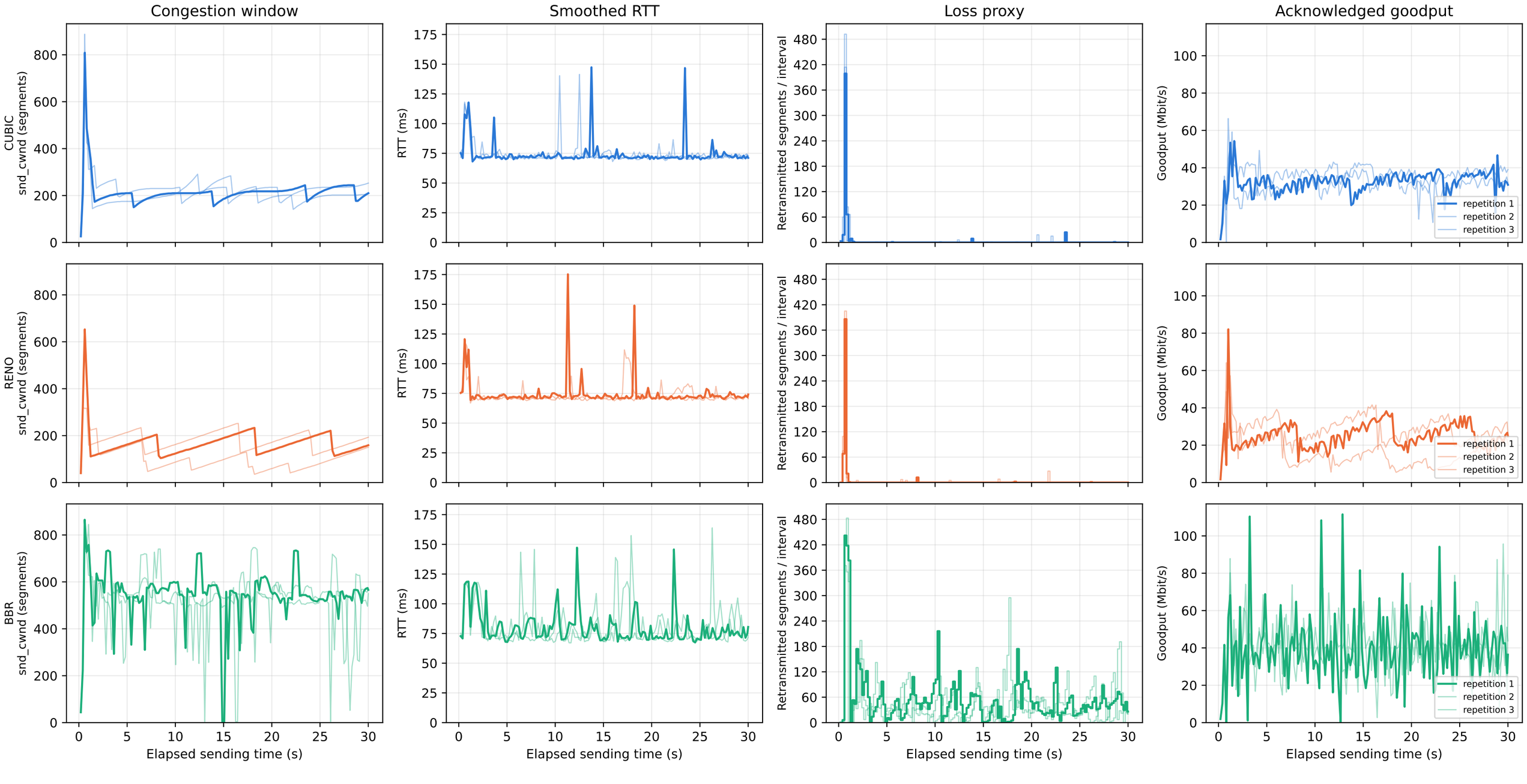
upload) succeeded on retry. Destinations: Los Angeles (`speedtest.lax12.us.leaseweb.net:5207`, min RTT ≈ 50–65 ms), Salt Lake City (`speedtest.xmission.com:5205`, ≈ 50 ms), `213.158.175.240:5201` (≈ 160 ms), `iperf.pendc.com:5202` (≈ 165 ms), Hong Kong (`speedtest.hkg12.hk.leaseweb.net:5209`, ≈ 210 ms). Host: Ubuntu 26.04.1 (ARM64) live session in a UTM virtual machine on an Apple M3 Mac, kernel `7.0.0-30-generic`, Python 3.14.4, run natively (not in Docker). BBR was loaded with `modprobe tcp_bbr` and added to `tcp_allowed_congestion_control`; the default stayed `cubic`. The VM uses UTM's Shared Network (NAT), which forwards packets and does not re-terminate TCP: MSS is 1448 bytes, unlike Docker Desktop's 65483. Other traffic on the home network was not controlled.

Derived per-run metrics (`comparison.py`, `run_metrics`):

```
min_rtt         = min(tcpi_min_rtt)          # raw kernel minimum; ≈ propagation delay
queueing_delay  = median(srtt) - min_rtt     # standing queue the flow builds
bottleneck_rate = max(tcpi_delivery_rate)    # best delivery rate observed
BDP             = bottleneck_rate × min_rtt  # bytes the path holds without queueing
cwnd_to_bdp     = median(cwnd) × MSS / BDP   # > 1: excess window sits in a queue
```

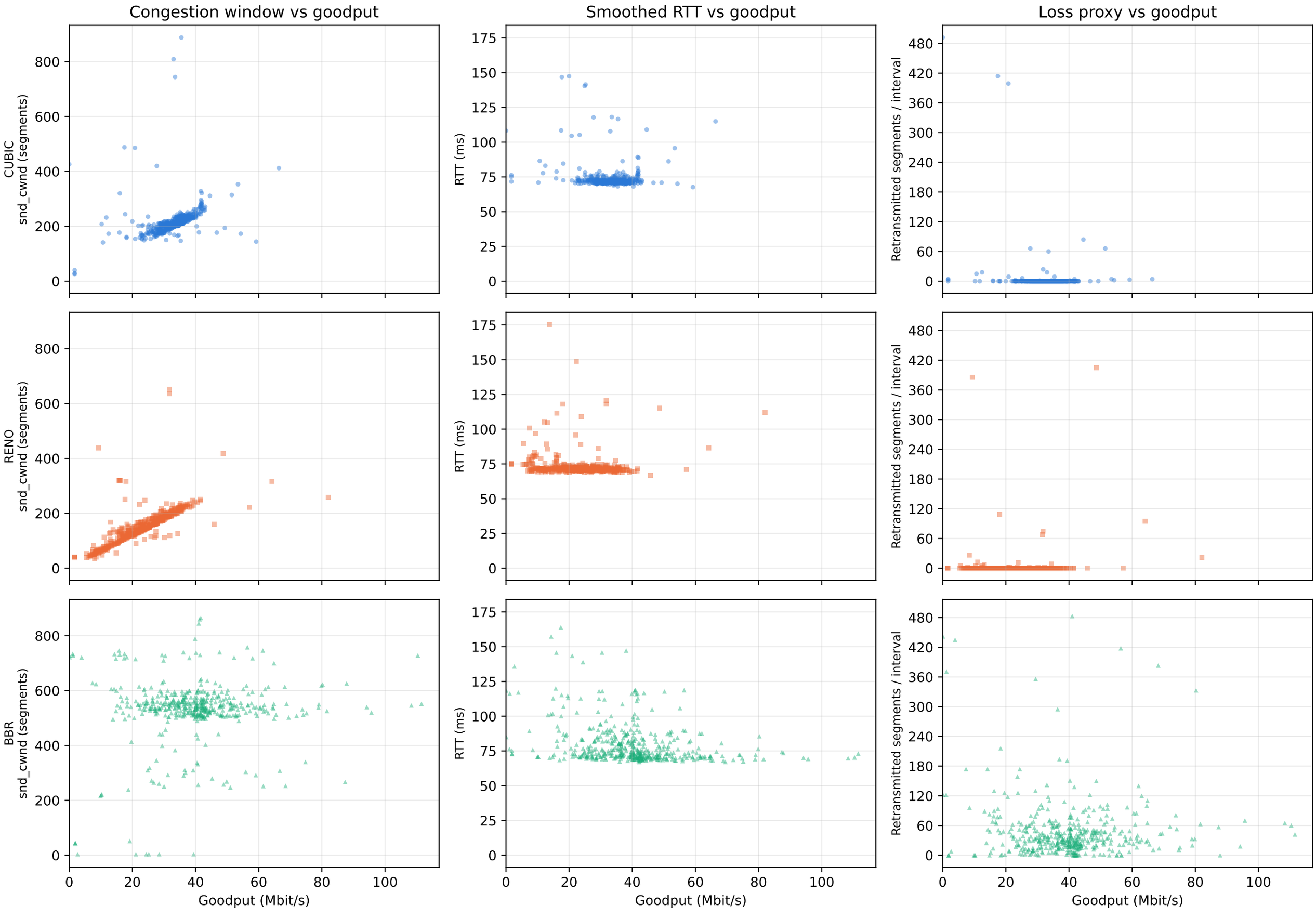
Plots to embed: q3-timeseries.pdf and q3-scatter.pdf (representative destination `speedtest.lax12.us.leaseweb.net:5207`), q3-compare.pdf (all runs), goodput.pdf; tables summary.csv, q3-algorithms.csv, q3-variability.csv, q3-runs.csv.

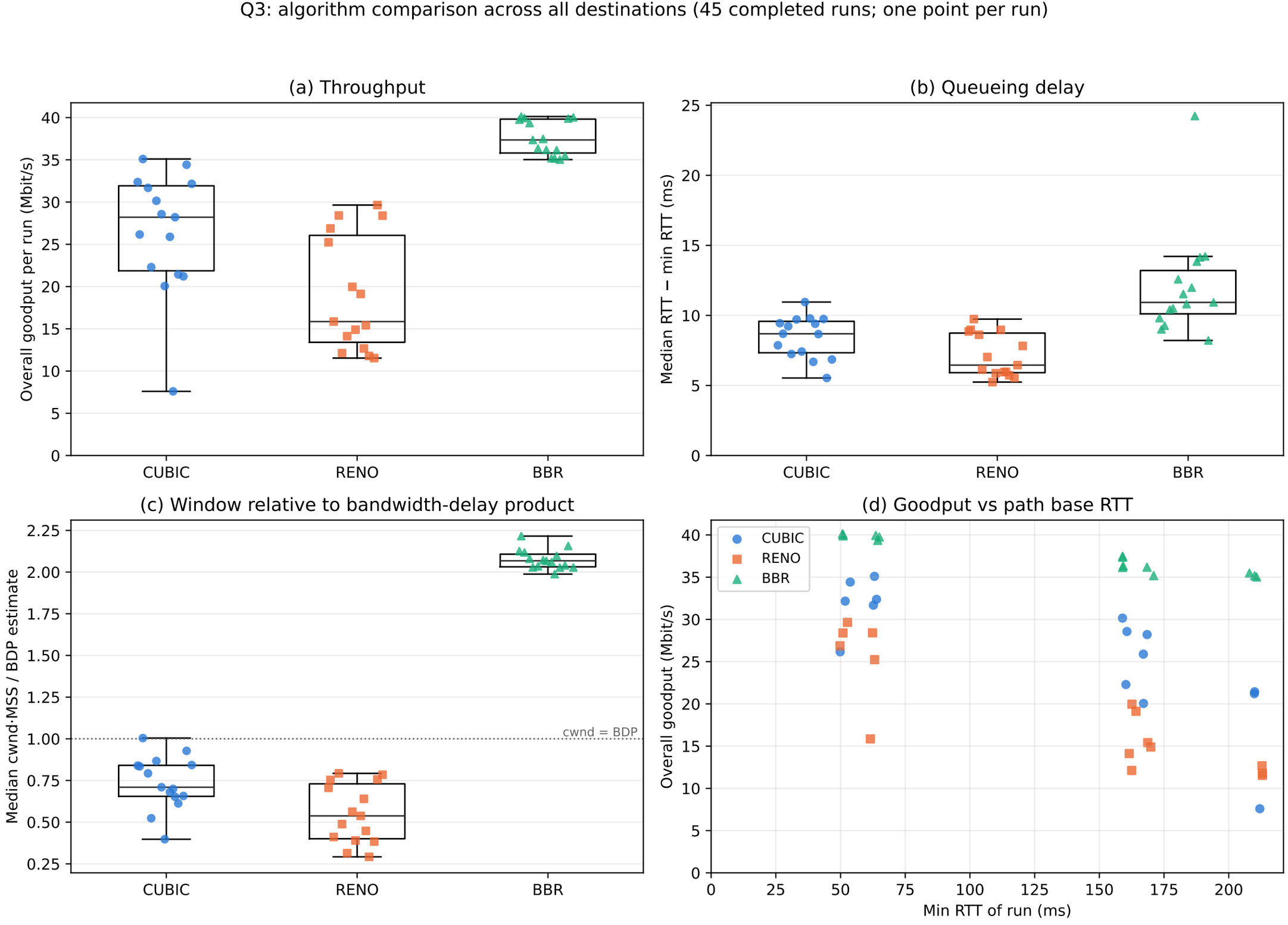
Q3: TCP statistics by algorithm, speedtest.lax12.us.leaseweb.net:5207
Each column shares one y-scale; first repetition bold, later repetitions faint



cwnd/RTT are endpoint snapshots; goodput/loss cover the preceding interval.

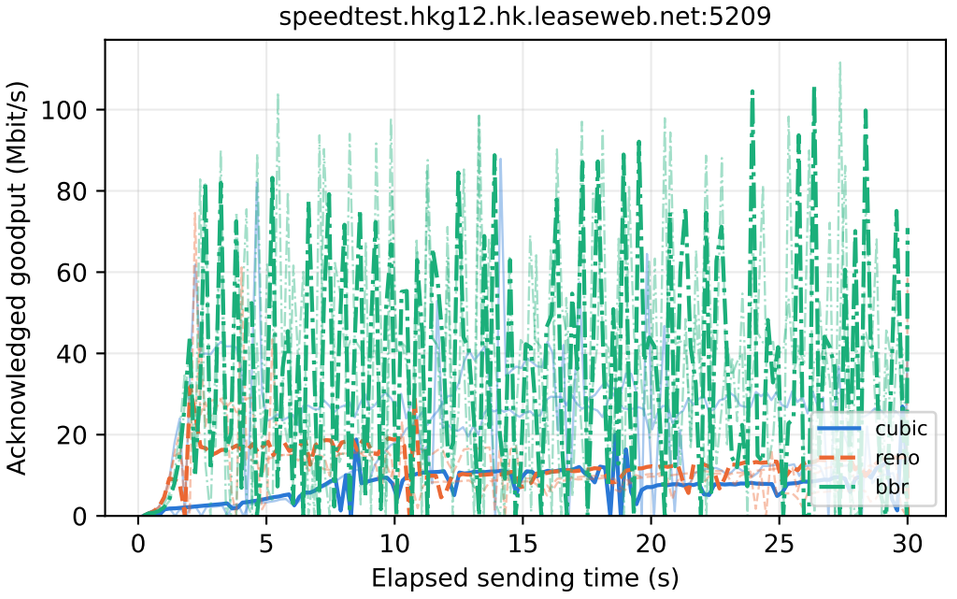
Q3: TCP metrics and goodput by algorithm, speedtest.lax12.us.leaseweb.net:5207
All repetitions pooled; each column shares axes. Correlation does not establish causation.

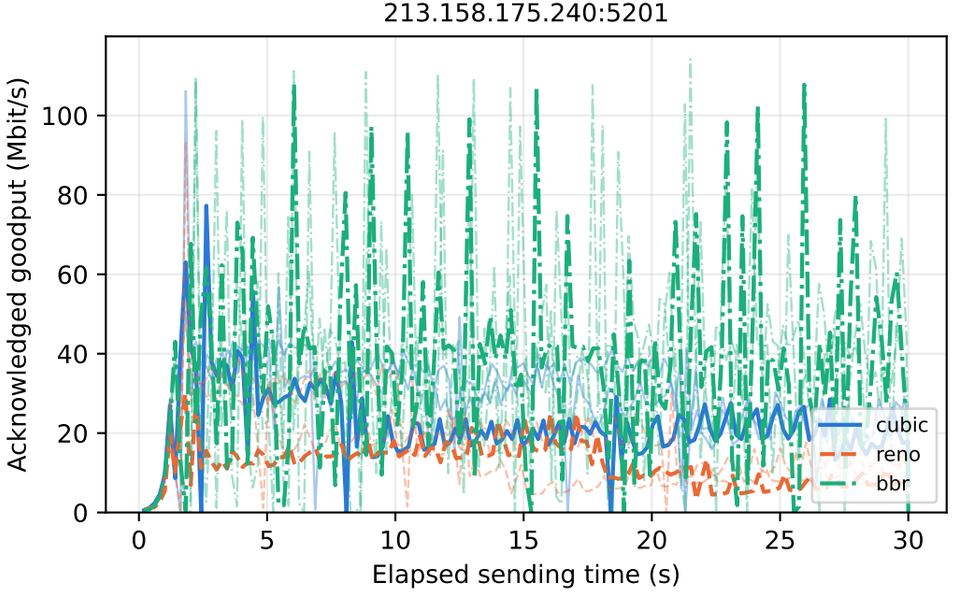


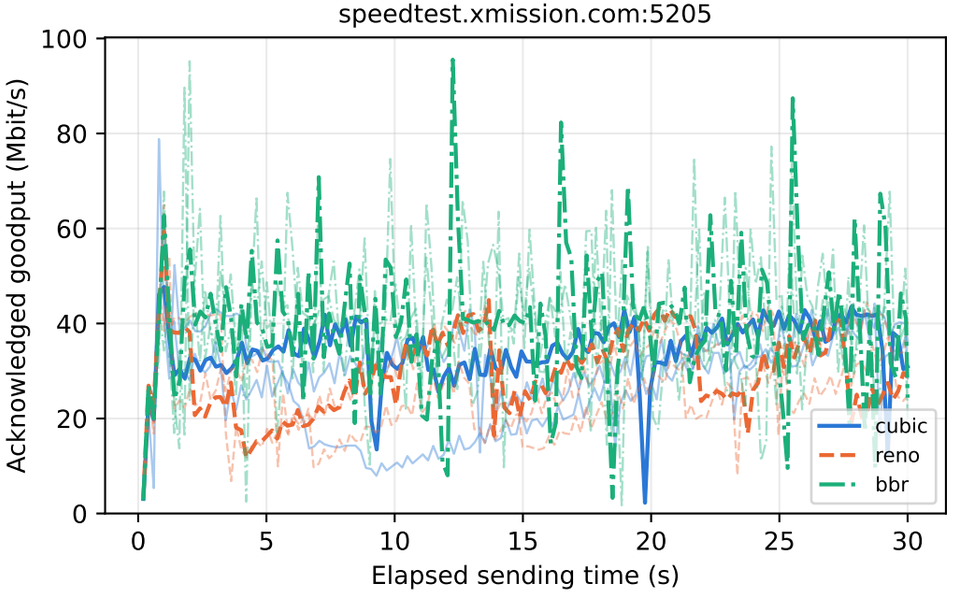

Q3: algorithm comparison across all destinations (45 completed runs; one point per run)
(a) Throughput
Overall goodput per run (Mbit/s)
0
5
10
15
20
25
30
35
40
CUBIC
RENO
BBR
(b) Queueing delay
Median RTT − min RTT (ms)
0
5
10
15
20
25
CUBIC
RENO
BBR
(c) Window relative to bandwidth-delay product
Median cwnd·MSS / BDP estimate
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00
2.25
cwnd = BDP
CUBIC
RENO
BBR
(d) Goodput vs path base RTT
CUBIC
RENO
BBR
Overall goodput (Mbit/s)
0
5
10
15
20
25
30
35
40
0
25
50
75
100
125
150
175
200
Min RTT of run (ms)
BDP estimate = max kernel delivery rate × min RTT. Public paths change between runs.

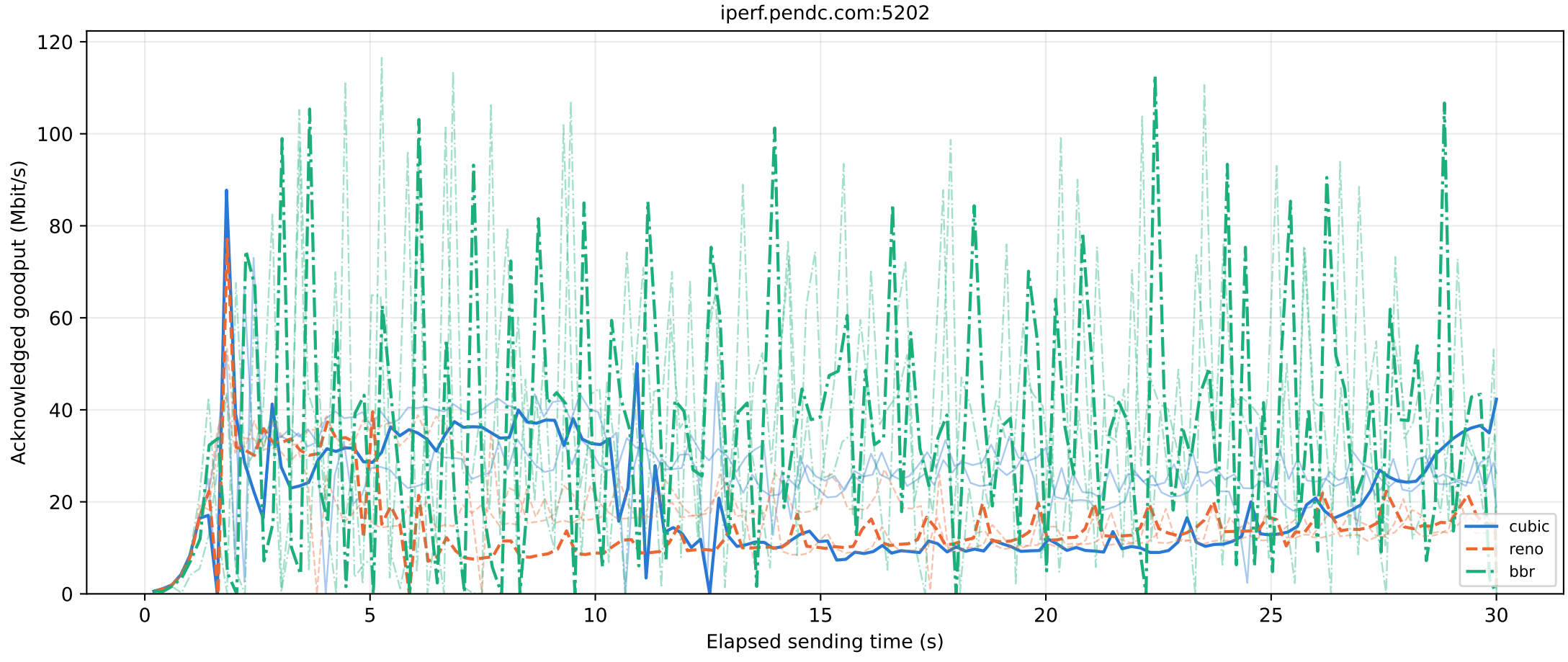
Q3 goodput: 5/5 destinations, 45 completed runs
first repetition bold; later repetitions faint










Q3 goodput: 5/5 destinations, 45 completed runs
first repetition bold; later repetitions faint
iperf.pendc.com:5202
Acknowledged goodput (Mbit/s)
Elapsed sending time (s)
0
20
40
60
80
100
120
0
5
10
15
20
25
30
cubic
reno
bbr

Hypotheses before the run:

	Expected throughput	Expected RTT/queue	Why
Reno	Lowest on high-BDP (long-RTT, fast) paths; similar on short paths	Fills buffer, sawtooth	+1 MSS per RTT takes BDP/MSS RTTs to fill the pipe; halves on loss
CUBIC	High on high-BDP paths	Fills buffer; RTT rises toward a loss	Growth depends on time since loss, not RTT; ×0.7 on loss
BBR	High on lossy or shallow-buffer paths	Near min RTT except periodic probes	Paces at measured bottleneck rate × min RTT; ignores isolated loss; may retransmit more

Results (medians over 15 runs per algorithm, from `q3-algorithms.csv`; retransmission percentages are retransmitted segments ÷ acknowledged segments):

	Goodput (Mbit/s)	Queueing delay (ms)	Retransmitted	Steady-state retransmitted (after 3 s)	cwnd·MSS / BDP	Best goodput / lowest RTT
CUBIC	28.2	8.7	0.65%	0.11%	0.71	0 / 1 of 5 destinations
Reno	15.9	6.5	0.75%	0.06%	0.54	0 / 4
BBR	37.4	10.9	7.8%	6.6%	2.07	5 / 0

Mean goodput per destination (`summary.csv`, Mbit/s, ordered by min RTT):

Destination	min RTT	CUBIC	Reno	BBR
Salt Lake City (xmission)	≈ 50 ms	30.9	28.3	40.0
Los Angeles (leaseweb)	≈ 63 ms	33.1	23.2	39.7
213.158.175.240	≈ 160 ms	27.0	15.4	37.1
iperf.pendc.com	≈ 165 ms	24.7	16.5	35.8
Hong Kong (leaseweb)	≈ 210 ms	16.7	12.0	35.2

Observations.

- *One shared, shallow bottleneck near the sender.* Every destination, from 50 ms to 210 ms away, tops out at the same ≈ 45–49 Mbit/s peak delivery rate, and BBR reaches 35–40 Mbit/s everywhere. The limit is therefore on the common part of the path, most likely the home upload link (or the VM's NAT), not at the servers. The buffer in front of it is small: median queueing delay stays at 5–11 ms for all three algorithms, and even CUBIC's p95 RTT is only ≈ 15 ms above the minimum. At 45 Mbit/s that is roughly 40–60 full-size packets, far less than the path BDP (≈ 200 packets at 50 ms, ≈ 800 at 210 ms). This makes the experiment close to a controlled one: the bottleneck stays the same and only the RTT changes between destinations.
- *CUBIC and Reno are loss-limited below the BDP.* The time series show the textbook sawtooth: Reno halves cwnd and climbs back linearly; CUBIC cuts to ≈ 0.7× and regrows along its concave-then-convex

curve. Each drop happens when cwnd reaches about BDP plus the small buffer, so after a cut the window is well below the BDP and the link sits partly idle until it regrows. The median window is only 0.71 (CUBIC) and 0.54 (Reno) of the BDP. With a buffer much smaller than the BDP, a loss-based flow cannot keep the pipe full.

- *The RTT penalty matches theory.* Reno grows by 1 MSS per RTT, so its recovery time grows with RTT and its goodput falls from 28 Mbit/s at 50 ms to 12 Mbit/s at 210 ms (panel d of `q3-compare.pdf`), as the Mathis model (rate ∝ MSS / (RTT·√p)) predicts. CUBIC's growth depends on the time since the last loss rather than on RTT, so it degrades less (31 → 17 Mbit/s). On average it beats Reno at every destination, by the most on the long paths.
- *Most CUBIC/Reno losses come from slow start.* 83% (CUBIC) and 92% (Reno) of their retransmissions happen in the first 3 s, when slow start overshoots the small buffer (the spike of ≈ 400 retransmitted segments per interval in the loss column). In congestion avoidance only 0.06–0.11% of segments are retransmitted: a few loss episodes per 30 s.
- *BBR keeps the pipe full by ignoring loss.* BBR's window sits at 2.0–2.1 × BDP in every run, exactly its in-flight cap (cwnd gain 2). The extra BDP worth of data cannot fit in a 40–60 packet buffer, so BBR overflows it continuously: 5–16% of segments are retransmitted, about 60 times CUBIC's steady-state rate. Because BBR's rate comes from its bandwidth estimate, not from loss, it keeps sending at ≈ 38 Mbit/s regardless of RTT and wins goodput at all five destinations. The cost is the highest queueing delay (median 10.9 ms, p95 ≈ 49 ms above the minimum against ≈ 13–15 ms for CUBIC/Reno) and the retransmissions. Its per-interval goodput is spiky (0–110 Mbit/s at 0.2 s) because retransmitted holes are filled in bursts and cumulative ACKs jump. In the LAX trace, the cwnd drop to ≈ 4 packets near 15 s is BBR's PROBE_RTT, which drains the queue to re-measure the minimum RTT.
- *Regimes.* On this path BBR operates at its rate and in-flight limits, with loss as a side effect. CUBIC and Reno operate in a loss-driven sawtooth below the BDP, and their throughput is set by RTT and by how often the small buffer overflows. Queueing delay is low for all three because the buffer is small; the hypothesis that loss-based algorithms would inflate RTT more than BBR does not hold here. With a deep buffer, CUBIC and Reno would fill it and build the larger queue.
- *Anomaly.* In repetition 1 to Hong Kong, CUBIC reached only 7.6 Mbit/s: the window grew slowly from the start and never reached the BDP, unlike repetitions 2–3 (21 Mbit/s). It ran right after a failed attempt to the same server. This one run accounts for most of CUBIC's spread at that destination.

Variability and limitations. BBR is very repeatable: the coefficient of variation of per-run goodput is 0.3–1.7% at every destination. CUBIC (5–47%) and Reno (5–28%) vary much more, because a loss episode at an unlucky moment costs a whole sawtooth period in a 30 s test. Public paths share the bottleneck with unknown cross traffic, routes and server load change between runs, and some servers may cap rates, so a difference smaller than the run-to-run spread is not evidence that one algorithm is better. Here BBR's lead over CUBIC exceeds the spread at every destination. CUBIC's lead over Reno does too, except at Salt Lake City (the shortest path, where both are close to the link rate) and Hong Kong (where the anomalous 7.6 Mbit/s CUBIC run widens CUBIC's spread). The delivery-rate maximum overstates the bottleneck slightly (ACK bursts can push it above the link rate), so cwnd/BDP ratios are approximate. Each algorithm's BDP is computed from its own run; BDP from the destination-wide maximum gives even lower ratios for Reno (≈ 0.35) and CUBIC (≈ 0.66). The bottleneck is probably the home network and VM NAT rather than the wide-area path, so the results describe a shallow-buffer, ≈ 45 Mbit/s access link. Interleaving and rotation reduce time bias but do not remove it. One upload stream per test means no algorithm competes with another of its own kind here, so fairness is not measured.

Q3(b): DART, a delay-aware, rate-targeting controller

Idea. Q3(a) shows the two failure modes DART is meant to avoid. CUBIC and Reno use loss as their only signal. On a buffer much smaller than the BDP, every loss leaves them below the BDP: their median window was 0.54–0.71 × BDP, and Reno lost more than half its goodput on the 210 ms path. Loss is also sparse, about 0.1% of segments in congestion avoidance, so it tells them little. BBR ignores loss and keeps 2 × BDP in flight, which fills the pipe but overflows the small buffer, causing 5–16% retransmissions and the highest queueing delay. DART targets the point between them: **the BDP plus a few packets**. It uses **queueing delay** as its main signal, **delivery rate** to size the window against the BDP, and **loss** only for backoff, scaled by whether the queue was actually full. Every input is a field we already decode from `TCP_INFO`: `rtt`, `rttvar`, `min_rtt`, `delivery_rate`, `total_retrans`, `snd_cwnd`.

```
constants: α = 2, β = 4 packets (target backlog in the bottleneck queue)
state:     cwnd, mode ∈ {STARTUP, CRUISE, COMPETE}
           min_rtt = windowed min RTT over 10 s
           btl_bw  = windowed max delivery_rate over 10 RTTs

once per RTT:
  bdp    = btl_bw × min_rtt / MSS                    # packets the pipe holds
  queued = cwnd × (1 − min_rtt / srtt)               # packets sitting in the queue
  noisy  = rttvar > 0.5 × (srtt − min_rtt)           # delay too jittery to trust

  STARTUP:
    cwnd = 2 × cwnd
    if btl_bw grew < 25% for 3 rounds or queued > β or loss:
      cwnd = bdp + α;  mode = CRUISE                 # pipe full: drop the startup queue

  CRUISE:
    if   queued < α and btl_bw still rising: cwnd += 2   # throughput says room left
    elif queued < α:                         cwnd += 1
    elif queued > β and not noisy:           cwnd -= 1
    cwnd = min(cwnd, bdp + 4β)                            # bound the standing queue
    if queued > β for 8 RTTs despite decreases:           # a loss-based flow fills the
      mode = COMPETE                                      #   buffer; backing off only starves us

  COMPETE:
    cwnd += 1                                          # Reno-style additive increase
    if queued < α for 4 RTTs: mode = CRUISE            # competitor gone

  if min_rtt not refreshed for 10 s:                   # re-measure the empty-queue RTT
    cwnd = 4 for max(200 ms, 1 RTT)

on a loss episode (once per recovery, from Δtotal_retrans > 0):
  if srtt − min_rtt > 2 × rttvar:  cwnd = max(0.7 × cwnd, 4)     # queue was full: congestion
  else:                            cwnd = max(0.9 × cwnd, bdp)   # queue empty: likely random loss
on retransmission timeout:
  cwnd = 1;  mode = STARTUP
```

Design choices.

- *Delay first.* Keeping only α–β packets queued holds RTT near `min_rtt`, which Reno and CUBIC cannot do because they grow until the buffer overflows.
- *Throughput sets speed and the ceiling.* Exponential startup exits when the delivery rate stops growing (the pipe is full), and the cap `cwnd ≤ BDP + 4β` bounds the queue even if the delay signal is wrong. The cap is additive, not BBR's 2 × BDP: in Q3(a), 2 × BDP meant 200–800 extra packets against a buffer of

about 40–60, and BBR retransmitted 5–16% of its segments. The windowed-max delivery rate tends to overestimate the BDP slightly, so the cap does not cut below the true BDP.

- *Loss is classified by delay.* A loss with an empty queue is probably random (wireless, policing), so DART cuts only 10% and never below the BDP. That avoids the throughput collapse loss-based algorithms suffer on lossy long paths. A loss with a full queue is congestion, so it cuts by 30% like CUBIC.
- *Dead-band and noise guard.* No change inside α–β, and no decrease when `rttvar` dominates the queueing signal, to avoid reacting to ACK jitter.
- *COMPETE mode.* Pure delay-based control (Vegas) is starved by CUBIC; a persistent queue that does not shrink when DART backs off means someone else is filling it.

Fairness. At a shared bottleneck all flows see the same queueing delay q. A flow at rate x keeps $\text{queued} = x \cdot q$ packets in the queue, and CRUISE holds queued in [α, β]. So every DART flow settles at a rate in [α/q, β/q], **independent of its RTT**, unlike Reno, whose share scales as 1/RTT. The dead-band means rates are equal only to within a factor β/α = 2. A single target (α = β, as in FAST TCP) gives exact proportional fairness but oscillates more. In COMPETE mode DART is additive-increase/multiplicative-decrease, which converges to equal shares among DART flows (Chiu–Jain). Against CUBIC or BBR it is Reno-like and is expected to get *less* than a fair share.

Convergence and stability. In CRUISE, ±1 packet per RTT with a dead-band reaches a fixed point at about BDP + α…β packets and stops moving, with no sawtooth. With N flows the queue holds about N·α…N·β packets, so delay grows with the number of flows. The 10 s min-RTT refresh drains the queue briefly; flows that see the new minimum reset their timers together, which tends to synchronize later refreshes (as in BBR).

Assumptions. One FIFO bottleneck; RTT changes come mainly from forward-path queueing; the kernel delivery-rate estimate is not application-limited (true for a bulk upload); min RTT changes slowly compared with the 10 s window.

Limitations.

- *Base-RTT error:* a flow that starts while a queue already exists measures an inflated `min_rtt`, underestimates its backlog, and takes more than its share until the next refresh. A route change has the same effect.
- *Shallow buffers blur the loss classifier.* In Q3(a) the buffer overflowed at only about 10–15 ms of queueing, which is comparable to RTT jitter. On such a link a congestive loss can look "random" and get the mild 0.9× cut. Floored at the BDP, that is the right response for a single flow, but with competing flows it backs off too little.
- *Reverse-path congestion and ACK compression* inflate RTT without a forward queue, so DART backs off unnecessarily.
- *Coexistence:* COMPETE is Reno-like, so DART loses against CUBIC on high-BDP paths and against BBR's pacing.
- *Parameters* (α, β, thresholds, 10 s windows) are chosen by reasoning, not tuned.
- *Not implemented or tested.* This is pseudocode; nothing here measures DART. A real test would need a kernel module or BPF `struct_ops` controller and a controlled bottleneck where competing flows can be measured.

AI acknowledgement

We used Codex (OpenAI) and Claude (Anthropic, through Claude Code) as assistants.

- **Q1/Q2:** Codex helped implement and review the iPerf3 protocol exchange (`iperf_client.py`), TCP_INFO sampling and goodput calculations (`tcp_stats.py`), and the experiment runner, summary tables, and plots (`experiment.py`, `reporting.py`). It helped check public-server uploads and saved interval calculations, and draft the metric explanations and trace observations.
- **Q3 code:** it helped write the CUBIC/Reno/BBR schedule with rotated order, repetitions, retries, and pauses (`experiment.py`), the per-algorithm pre-flight check (`tcp_stats.py`), and the comparison tables and plots (`comparison.py`).
- **Measurements:** it helped set up the Ubuntu VM used for all final runs, and diagnose why the earlier Docker Desktop data was invalid (its proxy ended TCP locally).
- **Analysis and writing:** it helped interpret the measurements, draft the Q1, Q2, and Q3 observations, and draft the DART design (pseudocode, fairness, convergence, assumptions, and limitations).